Dependently Typed World Models

Toward Proof-Carrying Learned Simulators

Anonymous

Submitted for review

August 2026

Abstract

World models are the internal working pictures through which many modern learning systems rehearse possible futures. They compress experience, imagine consequences, and help an agent choose before it acts. Yet their most important commitments—what counts as a legitimate state, which fragments of evidence may be combined, and what authority an imagined plan presupposes—are usually left implicit. This paper argues that dependent type theory offers a way to make those commitments explicit and machine-checkable. We introduce the notion of a *proof-carrying world model*: a learned simulator whose states and transitions arrive with evidence that they satisfy the conditions under which they may be used. To make the idea intuitive, we begin with a simple construction in which a world description is assembled from compatible local patches, much as a map is assembled from non-conflicting survey notes. Partial commutative monoids (PCMs) then appear as the resource-sensitive sharpening of that picture, giving the Rocq proof assistant a precise language for saying when pieces of a model may be joined and when they must remain apart. We also relate this view to Joint Embedding Predictive Architecture (JEPA), whose representations are likewise built from agreement across partial views rather than exhaustive reconstruction. The result is a research program in which learned world models become not only predictive, but also explicit about the structure, scope, and admissibility of what they claim to know.

Keywords: world models, dependent type theory, Rocq, partial commutative monoids, model-based reinforcement learning, proof-carrying programs, alignment.

1. Introduction

One of the most consequential ideas in artificial intelligence is that an agent need not meet the world only in the instant. It may, before acting, carry an inner stage on which candidate futures can be rehearsed. In model-based reinforcement learning, that stage is the world model: a learned internal simulator that compresses experience into a latent state and uses that state to forecast what actions might bring about [Sutton, 1990, Ha and Schmidhuber, 2018, Hafner et al., 2020, Schrittwieser et al., 2020, Hafner et al., 2023, Ye et al., 2021]. The practical appeal is easy to see. An agent that can imagine cheaply does not need to experiment as recklessly in reality.

The idea has also widened. In one lineage, world models are used explicitly for planning. In another, adjacent lineage, predictive systems such as Joint Embedding Predictive Architecture (JEPA) learn latent representations by asking what one partial view of a situation should allow a system to say about another [LeCun, 2022, Assran et al., 2023]. These models do not insist on reconstructing every pixel or every sensory detail. They aim instead for a more selective kind of understanding: enough internal structure to support prediction, abstraction, and downstream decision-making.

And yet the central mystery remains curiously underdescribed. A latent state in most current systems is useful, trainable, and often powerful, but semantically reticent. It carries little machine-checkable explanation of what it is allowed to represent, which fragments of evidence may be combined into it, or what constraints its imagined futures are supposed to respect. These commitments, if they are written down at all, typically live outside the model—in design docs, policy layers, runtime checks, or after-the-fact audits.

This paper argues that dependent type theory offers a way to move some of that structure inward. In a dependently typed setting, a value can travel together with evidence that it satisfies the conditions required for its use. Proof need not arrive as a bureaucratic seal applied later; it can accompany the object at the moment of construction. We draw in particular on the Rocq proof assistant [The Coq/Rocq Development Team, 2024] and on the FCSL-PCM library [Nanevski et al., 2014], which develops partial commutative monoids (PCMs) as a disciplined language for describing how separable resources may and may not be composed.

Before reaching PCMs, however, there is a gentler picture. One can think of a world model as something assembled from local patches: a few observations here, a capability token there, a short trace of what happened when an action was taken. Some patches fit together; some contradict one another; some describe different parts of a larger whole. This patchwork intuition is simple enough to grasp in ordinary language, yet it already begins to sound like the latent geometry of JEPA, where partial views are asked to predict one another, and like the formal discipline of Rocq, where admissible constructions must be stated explicitly. Our claim is that these are not isolated ideas but adjacent ways of describing the same aspiration: a learned internal model that is both predictive and clear about the terms on which its pieces fit.

Our contributions are:

1. A conceptual framework—the *proof-carrying world model*—that articulates how dependent types can annotate the operational interfaces of a learned simulator (§3).
2. A gentler mathematical bridge from compatible local patches to PCMs, showing how an intuitive gluing picture can mature into a resource-aware semantics for agent histories, capability scopes, and latent predictive structure (§4).
3. Small Rocq formalizations illustrating these ideas, from basic proof-carrying state through typed rollout traces to heap-style ownership of sensor boundaries (§5).
4. A discussion of research directions, including explicit connections to JEPA-style predictive learning, alignment-aware planning, and multi-agent trust (§6).

We do not claim a complete implementation. We offer a research direction and a working vocabulary for future development.

2. Background

2.1. World Models in Reinforcement Learning

A world model, in the narrow reinforcement-learning sense, is a learned surrogate for the dynamics of an environment. Formally one may write it as a map $f_\theta : \mathcal{Z} \times \mathcal{A} \to \mathcal{Z} \times \mathcal{R}$, sending a latent state and an action to a predicted successor state and reward. But that notation only hints at the broader intuition. A world model is a compressed, internal picture of how things change: what tends to follow what, which possibilities remain open, and which imagined futures are worth spending computation on.

The modern lineage is familiar. Dyna first made explicit the advantage of learning and planning in tandem [Sutton, 1990]. Ha and Schmidhuber's World Models popularized the image of an agent dreaming inside its own latent space [Ha and Schmidhuber, 2018]. Dreamer and its successors showed that long-horizon imagination could become a practical engine for control [Hafner et al., 2020, 2021, 2023]. MuZero and EfficientZero demonstrated that one need not reconstruct the visible world in full detail in order to plan effectively [Schrittwieser et al., 2020, Ye et al., 2021].

JEPA extends the same family resemblance in a slightly different direction. Rather than learning to regenerate an entire input, a JEPA-style system learns to predict the representation of a withheld target view from an available context view [LeCun, 2022, Assran et al., 2023]. What matters is not visual fidelity but predictive sufficiency: the latent state should preserve the structure needed to say something reliable about what lies beyond the current glimpse. In that sense, JEPA is not a departure from the world-model story so much as a sharpening of one of its oldest instincts.

Across these traditions, a recurring tension remains between *predictive skill* and *explicit meaning*. A model may be extraordinarily good at forecasting what happens next and still be silent about what sort of state it is claiming to inhabit,

what sources of evidence it has mixed together, and what operational limits should govern its use. This gap is the starting point for our proposal.

2.2. Dependent Type Theory and the Rocq Proof Assistant

Dependent type theory [Martin-Löf, 1984, Coquand and Huet, 1988] begins from a simple but powerful thought: the description of an object may be allowed to mention the object itself. Instead of saying merely "this is a vector," one may say "this is a vector of length n," and insist that n be part of the object's type. A familiar example is $\mathsf{Vec}(n)$, the type of vectors of exactly n elements. More generally, a value of type $\{x : A \mid P(x)\}$ is both a value in A and evidence that it satisfies $P(x)$.

For readers outside formal methods, the philosophical point matters more than the notation. In an ordinary program, one often constructs a value first and checks later whether it meets the relevant conditions. In a dependently typed program, one can require those conditions at birth. Some classes of mismatch never become runtime problems because the system refuses to construct the wrong object in the first place.

Rocq (formerly Coq) [The Coq/Rocq Development Team, 2024] implements the Calculus of Inductive Constructions [Coquand and Huet, 1988, Paulin-Mohring, 2015], a rich dependent type theory supporting:

- *Inductive types* and pattern matching;
- *Dependent records* (`Record`) that bundle values with their correctness conditions;
- *Proof terms* as first-class citizens of the same language as programs;
- *Extraction* to OCaml or Haskell, enabling certified programs to run outside the proof environment.

The MathComp library [Gonthier et al., 2013] and the FCSL-PCM library [Nanevski et al., 2014] provide mature algebraic foundations—including Boolean reflection, finite maps, and partial commutative monoids—on which we build.

2.3. Compatible Patches Before Algebra

Before introducing PCMs, it helps to start with a more ordinary picture. Suppose a world is not given all at once, but only in pieces: a short sensor trace, a geometric hint about a hidden object, a note that a certain action is currently permitted. The simplest mathematical way to model such fragments is as *finite partial maps*: small tables that assign values only where the agent currently knows something.

Definition 2.1 (Compatible patch family)**.** *Let I be an index set and V a set of values. A* patch *is a finite partial function $p : I \rightharpoonup V$. Two patches p and q are* compatible *when they agree wherever both are defined, that is,*

$$\forall i \in \mathrm{dom}(p) \cap \mathrm{dom}(q),\ p(i) = q(i).$$

When p and q are compatible, their glued union $p \sqcup q$ is the obvious combined partial map on $\mathrm{dom}(p) \cup \mathrm{dom}(q)$.

This is the mathematics of an atlas, a notebook, or a map made from local surveys. A larger picture can be assembled when the smaller pictures do not contradict one another where they meet. Category theorists would recognize a sheaf-like gluing intuition in the background, but the concrete finite-map version is already enough for our purposes. It gives a first-principles answer to the question "what does it mean to build a world model from scratch?" One begins not with a monolithic state, but with admissible pieces that can be assembled into one.

2.4. Partial Commutative Monoids

Definition 2.2 (Partial Commutative Monoid)**.** *A* partial commutative monoid *(PCM) is a set U equipped with a distinguished element $\mathbf{0} \in U$, a* validity *predicate* $\mathsf{valid} : U \to \mathsf{Prop}$*, and a partial binary operation $\oplus : U \to U \to U$ satisfying:*

commutativity :	$x \oplus y = y \oplus x$
associativity :	$(x \oplus y) \oplus z = x \oplus (y \oplus z)$
unit :	$x \oplus \mathbf{0} = x$
validity domain :	$\mathsf{valid}(x \oplus y) \Rightarrow \mathsf{valid}(x) \wedge \mathsf{valid}(y)$

When $\mathsf{valid}(x \oplus y)$ *fails, $x \oplus y$ returns a distinguished* undefined *element* Undef*.*

The easiest way to read this definition is as a resource-sensitive refinement of the patch picture above. Compatible patches asked only whether two local descriptions could coexist. A PCM asks the stricter question that software and agents often face in practice: can these fragments be *jointly held or used* without conflict? In some settings, overlap with agreement is harmless; in others—ownership, authority, leased capabilities, authored histories—overlap itself is exactly what must be controlled.

PCMs arose in the context of separation logic [Reynolds, 2002, O'Hearn, 2019] as the canonical semantic domain for *separating conjunction*: two resources may be combined precisely when their join is valid. The FCSL-PCM library [Nanevski et al., 2014] provides a rich Rocq hierarchy of PCM instances, including heap-like finite maps. We repurpose that discipline here, not because world models are literally heaps, but because the question of which fragments may be safely composed is central to both.

3. A Framework for Proof-Carrying World Models

3.1. Motivation

Most learned latent states are described, at bottom, as arrays of numbers. That description is enough for optimization, but it says very little about meaning. It does not say whether the state was assembled from compatible evidence, whether it presupposes a certain capability scope, or whether it is even the kind of state a planner should be allowed to imagine from. Those judgments are usually external to the model.

Our proposal is to make admissibility *intrinsic*. If a state is meant to stand for "a history built from non-conflicting observations under capability scope κ," then its type should say so. If a plan is meant to require certain permissions, the need for those permissions should be visible in the plan's construction. Scope confusion then ceases to be a policy bug discovered late; it becomes a type error discovered early.

3.2. Proof-Carrying State

We define a *proof-carrying state* as a dependent record bundling a representational value with a machine-checked admissibility witness.

Definition 3.1 (Proof-Carrying State)**.** *Let S be a set of representational states and let* $\mathsf{Adm} : S \to \mathsf{Prop}$ *be an admissibility predicate. A* proof-carrying state *is a pair (s, h) where $s \in S$ and $h : \mathsf{Adm}(s)$.*

The key property is psychological as well as technical. One is no longer asked to trust that a state has been blessed somewhere upstream. The blessing is part of the object. Construction and certification occur together, or not at all.

3.3. Proof-Carrying World Model Interface

A world model operating over proof-carrying states must be designed so that:

1. *Observation closure*: applying the observation operator to an admissible state produces an admissible state.
2. *Imagination validity*: applying the imagination (rollout) operator to an admissible state produces a valid (non-undefined) state.

These are not runtime checks but proof obligations discharged at definition time. A world model that cannot satisfy them is, in a precise sense, *ill-typed*—it cannot be instantiated.

The following Rocq sketch captures the interface:

```
From mathcomp Require Import ssreflect ssrbool ssrnat seq.
From pcm Require Import pcm.

Section TypedWorld.
Context {State : pcm}.

Record WorldModel := {
  observe  : State -> seq nat -> State;
  imagine  : State -> seq nat -> State;
  admissible : State -> Prop;
  observe_ok : forall s frame,
    admissible s -> admissible (observe s frame);
  imagine_ok : forall s actions,
    admissible s -> valid (imagine s actions)
}.

End TypedWorld.
```

Listing 1: Proof-carrying world model interface in Rocq

The `State` is parameterized by an arbitrary PCM, so the framework is not committed to any particular representation; heap-like finite maps, trace histories, and richer algebraic structures are all valid instantiations. A JEPA-style model can be read through the same interface: the observable context builds a typed state, the predictive step imagines a target representation, and the surrounding proofs state what kinds of context are sufficient for what kinds of prediction.

4. From Compatible Patches to Resource-Sensitive World Models

4.1. From Local Patches to History Composition

The compatible-patch picture gives us a gentle beginning: pieces of a world may be glued when they agree. But systems that act in the world often need a stricter discipline. A history should not contain two independently authored entries for the same moment unless one has said exactly how such conflicts are resolved. A capability scope should not silently duplicate authority. A leased resource should not be owned twice simply because two tables happen to list the same symbol.

This is where the PCM point of view earns its keep. Separation logic was designed to reason locally: a proof may concern one fragment of the heap, and the frame rule extends it to larger heaps the code does not touch [Reynolds, 2002]. The semantic engine underneath is that composition via $\oplus$ is defined only when the relevant fragments can be combined without a resource conflict. We propose an analogous discipline for world-model state.

An *observation history* may be represented as a finite map from timesteps to observation records, while a *capability scope* may be represented as a finite map from capability identifiers to authorization witnesses. These are still local pieces of a larger picture. The difference is that now we are not merely asking whether they tell a consistent story; we are asking whether they may be held together as a single admissible resource.

Definition 4.1 (History PCM)**.** *Let $\mathcal{T}$ be a countable set of timesteps and $\mathcal{O}$ a set of observation records. Define $\mathcal{H} = \mathcal{T} \rightharpoonup \mathcal{O}$ (partial functions with finite support). The PCM structure on $\mathcal{H}$ is:*

$$\mathsf{valid}(h) \triangleq h \neq \mathsf{Undef}$$

$$h_1 \oplus h_2 \triangleq \begin{cases} h_1 \cup h_2 & \text{if } \mathrm{dom}(h_1) \cap \mathrm{dom}(h_2) = \emptyset \\ \mathsf{Undef} & \text{otherwise} \end{cases}$$

This is exactly the PCM of disjoint-key finite maps, realized in Rocq by `natmap` *[Nanevski et al., 2014].*

Under this reading, combining two histories is only valid if they record distinct moments in time. Conflicting entries—two agents writing different observations for the same timestep—produce an undefined result. The point is not pedantry.

It is to refuse the quiet habit, common in learned systems, of allowing incompatible evidence to blur into a single latent state with no clear account of how the contradiction was handled.

4.2. JEPA and the Grammar of Partial Views

The connection to JEPA is closer than it first appears. A JEPA-style system is trained on the belief that one partial view of a situation should determine a useful prediction about another partial view [LeCun, 2022, Assran et al., 2023]. It is, in spirit, a theory of world understanding through fragments. One sees a portion of the scene, forms an internal representation, and asks what that representation licenses one to expect about what remains hidden.

The compatible-patch picture offers a mathematical idiom for this intuition. Context and target can be understood as partial descriptions of a larger world; the learned representation is valuable insofar as it preserves the structure that makes those descriptions cohere. The difference is that JEPA learns this coherence statistically, from data, whereas PCM-based reasoning states certain forms of coherence explicitly, in logic. The opportunity lies in the overlap. One can imagine a system in which learned joint embeddings supply compact predictive patches, while Rocq certifies when such patches may be composed, which scopes they presuppose, and what kinds of downstream planning claims they are allowed to justify.

Seen this way, PCM is not a rival to JEPA but a complement. JEPA says that a useful latent state need not reconstruct everything; it need only preserve what matters for prediction. PCM says that a useful latent state should also come with rules for composition: which fragments can be joined, which authorities travel with them, and when an attempted merge should simply be declared inadmissible.

4.3. Typed Rollout Traces

A *rollout trace* in our framework is a proof-carrying history: a finite map from timesteps to rewards, annotated with a positive-horizon witness. The following Rocq definition makes this concrete:

```
From mathcomp Require Import ssreflect ssrbool ssrnat seq.
From pcm Require Import pcm natmap.

Record rollout := {
  trace      : history nat;
  horizon    : nat;
  horizon_pos : horizon > 0
}.

Definition extend_trace (r : rollout) (t reward : nat) :=
  if t == 0 then None
  else if valid ((t \-> reward) \+ trace r)
       then Some {| trace      := (t \-> reward) \+ trace
```

```
          r;
                    horizon     := horizon r;
                    horizon_pos := horizon_pos r |}
      else None.
```

Listing 2: Typed rollout trace

The `extend_trace` function is partial: it returns `None` whenever the new timestep is zero (a sentinel for invalid time) or whenever the new entry conflicts with an existing entry in the map. The important point is conceptual. Partiality here is not a programming inconvenience but a faithful description of the mathematics. Some attempted extensions really do fail to produce a coherent world fragment, and the type should say so.

4.4. Capability Scopes as Resource Types

PCMs also model *capability scopes*: the actions an agent is currently authorized to perform. We represent a scope as a finite map from capability identifiers to authorization tokens. Two scopes may be combined only if they govern disjoint sets of capabilities. An agent attempting to act under a capability it does not hold faces a type error at the point of action construction, not a runtime permission denial.

This design clarifies a distinction that learned systems often blur: the difference between what a model predicts and what a system is entitled to do. JEPA-style prediction may furnish a powerful account of what latent structure is present; typed resource semantics can add a separate account of which actions that structure licenses. The two together suggest a world model that is both perceptive and disciplined.

5. Illustrative Rocq Formalizations

5.1. A Tiny Example of Proof-Carrying State

Before introducing PCM structure, we illustrate the simplest instance of proof-carrying state: a rollout record whose `horizon` field is certified by a length witness.

```
From Coq Require Import List Arith.
Import ListNotations.

Record CheckedRollout := {
  horizon         : nat;
  actions         : list nat;
  horizon_matches : length actions = horizon
}.

Definition single_action (a : nat) : CheckedRollout :=
  {| horizon         := 1;
     actions         := [a];
```

```
    horizon_matches := eq_refl |}.
```

Listing 3: Minimal proof-carrying rollout record

The field `horizon_matches` is not a runtime assertion but a proof term. It is discharged by `eq_refl` at the point of construction, confirming that the singleton list `[a]` has length 1. A `CheckedRollout` with mismatched horizon cannot be constructed; the attempt would produce a type error. This is the paper's basic promise in miniature: if a state claims to summarize a horizon, that claim can be made part of the object rather than an informal comment about it.

5.2. Resource Ownership at the Sensor Boundary

We illustrate resource-typed sensor access using the `heap` PCM from FCSL-PCM. A sensor write is defined to return `Undef` if the target pointer is null, and a lemma certifies that non-null writes preserve validity.

```
From mathcomp Require Import ssreflect ssrbool.
From pcm Require Import heap.

Definition sensor_write (p : ptr) (v : dynamic id) (h :
   heap) :=
  if p == null then Undef else upd p v h.

Lemma sensor_write_preserves_nonnull p v h :
  p != null ->
  sensor_write p v h != Undef.
Proof.
move=> p_nonnull.
rewrite /sensor_write p_nonnull.
case: h=> //= fm pf.
by case: decP=> // /eqP.
Qed.
```

Listing 4: Heap-style ownership for sensor boundary

The lemma `sensor_write_preserves_nonnull` is a machine-checked guarantee. Any system that calls `sensor_write` on a non-null pointer inherits the proof; any downstream code that requires a valid heap fragment can accept the result directly. The larger moral is that world-model interfaces may benefit from the same style of explicit ownership accounting that made formal reasoning about mutable state practical.

5.3. Composing Histories Without Ambiguity

The following sketch demonstrates how two disjoint history fragments may be safely combined into a single rollout.

```
From mathcomp Require Import ssreflect ssrbool ssrnat.
From pcm Require Import pcm natmap.
```

```
(* Two disjoint observation fragments *)
Definition obs1 : history nat := 1 \-> 42.
Definition obs2 : history nat := 2 \-> 17.

(* Their join is valid because key sets are disjoint *)
Lemma obs_join_valid : valid (obs1 \+ obs2).
Proof. by rewrite /obs1 /obs2; compute. Qed.

(* An attempt to join conflicting entries produces Undef
   *)
Definition obs1' : history nat := 1 \-> 99.

Lemma obs_conflict_invalid : ~~ valid (obs1 \+ obs1').
Proof. by rewrite /obs1 /obs1'; compute. Qed.
```

Listing 5: Safe composition of disjoint observation fragments

These two lemmas capture the core safety property: valid composition is not assumed or checked at runtime; it is established once, by proof, and then inherited by any code that depends on the composed history. In the language of the earlier sections, we have moved from the image of compatible patches to a precise account of when compatible fragments count as a single admissible resource.

6. Discussion and Research Directions

6.1. Alignment-Aware Planning

The alignment problem for learned agents [Russell, 2019, Gabriel, 2020] is, in part, a problem of specification: how does an agent know that the actions it is considering are within the scope of what it is supposed to do? Current approaches rely heavily on reward shaping, RLHF [Christiano et al., 2017, Bai et al., 2022b], and constitutional constraints [Bai et al., 2022a], all of which are intrinsically extrinsic—they surround the model with evaluative signals rather than embedding constraints within its representational structure.

A proof-carrying world model offers a complementary approach. By requiring that high-stakes action types carry typed witnesses of their preconditions, the planning procedure is structurally prevented from producing plans that would violate stated constraints—not because a guard intercepts the output, but because the output type cannot be constructed without the required evidence. The difference is analogous to the difference between a runtime bounds check and a statically verified array access: the latter eliminates a class of error by construction, before execution begins.

We envision a planning module whose action type is parameterized by a capability scope κ: an action of type $\mathsf{Action}(\kappa)$ may only exercise the capabilities enumerated in κ. The planner's type signature then enforces that it cannot produce an action requiring capability c unless $c \in \kappa$. Scope escalation becomes a

type error.

The JEPA connection matters here as well. If a predictive model is trained to infer withheld structure from partial context, the question naturally follows: what kinds of context are sufficient for which kinds of action? Dependent types offer a way to state that question sharply. A latent prediction can be treated not as a free-floating hint, but as an object whose admissible uses are tied to the evidence from which it was formed.

6.2. Multi-Agent Trust and Fragment Provenance

In multi-agent settings, observations arrive from diverse sources with varying degrees of trust. A dependently typed world model can represent this by annotating observation histories with *provenance types*: a history of type $\mathsf{Hist}(\tau)$ was collected under trust level τ, and the type system can enforce that a planner operating under trust level τ' may only act on histories whose provenance satisfies $\tau' \preceq \tau$ for some trust ordering $\preceq$.

The PCM structure ensures that joining histories of incompatible provenance produces an `Undef` result rather than a silently untrusted composite. This makes cross-provenance contamination detectable at the type level—a property that has practical significance wherever agents interact with adversarially constructed inputs or aggregated data of heterogeneous origin.

6.3. Connections to Verified Machine Learning

Our proposal is broadly consistent with a growing literature on *verified machine learning* [Seshia et al., 2018, Katz et al., 2019, Liu et al., 2021] and *certified programs* [Leroy, 2009, Klein et al., 2009]. These works pursue machine-checked correctness for specific properties—robustness bounds, absence of undefined behavior—but typically treat the neural component as a black box to be analyzed from the outside. Our contribution is to argue that the *interface* of a world model—its observation operators, imagination rollouts, and planning predicates—can itself be given a dependently typed specification that interoperates naturally with proof-assistant toolchains.

This is complementary to, rather than in competition with, techniques for verifying the *internals* of neural networks. A world model whose interface carries machine-checked guarantees can be composed with a certifying planner even if the neural weights themselves remain opaque. This point is especially suggestive for JEPA-style systems, whose learned embeddings are compact and predictive but not always semantically explicit. A typed interface offers a place where those embeddings may acquire precise obligations without forcing the entire learning procedure into symbolic form.

6.4. Toward a Full Formal Development

The formalizations in §5 are illustrative sketches, not a complete verified system. A full development would require:

1. A Rocq formalization of the world model interface parameterized by an arbitrary PCM, with proofs of the stated closure properties.
2. A typed patch algebra for context and target views, sufficient to model JEPA-style predictive interfaces alongside sequential rollout constraints.
3. A formalization of capability scopes as PCMs over finite maps of authorization tokens, with lifting lemmas connecting scope composition to the planning type.
4. An extraction pipeline enabling certified planning code to run inside a standard MBRL training loop.

We leave this development as future work, noting that each component has precedent: the FCSL-PCM library provides the algebraic foundations; verified functional programs with extraction are routine in Rocq; and typed planning interfaces have been explored in the context of verified task planners [Blanchette and Nipkow, 2010].

7. Related Work

World models. Ha and Schmidhuber's World Models [Ha and Schmidhuber, 2018] helped establish the "dream-then-act" paradigm for visual planning. The DreamerV1–V3 series [Hafner et al., 2020, 2021, 2023] demonstrated that continuous control and Atari tasks can be mastered through pure imagination. MuZero [Schrittwieser et al., 2020] showed that a latent model without ground-truth reconstruction suffices for search-based planning. EfficientZero [Ye et al., 2021] extended this to extreme sample efficiency. None of these architectures address typed invariants on the model interface.

JEPA and predictive representation learning. LeCun's program for autonomous machine intelligence [LeCun, 2022] argues that useful intelligence should be grounded in predictive world models that operate in latent space rather than through exhaustive reconstruction. I-JEPA [Assran et al., 2023] makes this idea concrete by learning to predict target embeddings from context embeddings. Our use of compatible patches and typed composition is meant to be complementary: JEPA supplies a modern account of how predictive latent structure may be learned, while Rocq and PCM-like semantics supply a way to say, with precision, what kinds of fragments may be combined and what such combinations authorize.

Formal methods and learning. Seshia et al. [Seshia et al., 2018] survey formal specification and verification techniques applicable to learning-enabled systems, emphasizing runtime monitors and assume-guarantee contracts. Neural network verification [Katz et al., 2019, Liu et al., 2021] focuses on robustness of fixed networks. CompCert [Leroy, 2009] and seL4 [Klein et al., 2009] demonstrate that large software systems can be given machine-checked correctness proofs; our work proposes that learned interfaces can participate in such developments.

Separation logic and resources. Reynolds [Reynolds, 2002] introduced separation logic. O'Hearn [O'Hearn, 2019] surveys two decades of development. Nanevski et al. [Nanevski et al., 2014] develop FCSL as a Hoare-logic framework for concurrent separation reasoning, with a mature Rocq library. We repurpose this infrastructure for compositional history reasoning.

Alignment and specification. Russell [Russell, 2019] frames alignment as a problem of misspecified objectives. Christiano et al. [Christiano et al., 2017] and Bai et al. [Bai et al., 2022b,a] address alignment through human feedback and constitutional constraints. Gabriel [Gabriel, 2020] provides a conceptual taxonomy. Our proposal is orthogonal: rather than specifying objectives more accurately, we propose to specify the operational interface of the model more precisely.

8. Conclusion

We have argued that world models become easier to reason about when they are understood as constructions rather than mysteries: first as local patches that must fit together, then as resource-sensitive objects whose composition must be governed explicitly. Dependent type theory gives this intuition a precise home. PCMs provide one especially useful discipline for stating when histories, capabilities, and latent fragments may be joined. JEPA, meanwhile, reminds us that modern predictive systems are already built around partial views and selective sufficiency rather than exhaustive reconstruction. The conceptual gap between these traditions is therefore smaller than it first appears.

The proposal is not that proofs replace learning. A well-typed world model may still be empirically wrong about the world. The proposal is narrower and, for that reason, perhaps more realistic: that learned internal models can be asked to carry clearer commitments about admissibility, composition, and use. If a system is going to imagine on our behalf, it is reasonable to ask that its imagination come with terms.

References

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023.

Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022a.

Yuntao Bai et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022b.

Jasmin Christian Blanchette and Tobias Nipkow. Nitpick: A counterexample

generator for higher-order logic based on a relational model finder. In *Interactive Theorem Proving (ITP)*, pages 131–146, 2010.

Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2–3):95–120, 1988.

Iason Gabriel. Artificial intelligence, values, and alignment. *Minds and Machines*, 30:411–437, 2020.

Georges Gonthier et al. A machine-checked proof of the odd order theorem. In *International Conference on Interactive Theorem Proving (ITP)*, pages 163–179, 2013.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations (ICLR)*, 2021.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Guy Katz, Derek A. Huang, Duligur Ibeling, et al. The marabou framework for verification and analysis of deep neural networks. In *Computer Aided Verification (CAV)*, pages 443–452, 2019.

Gerwin Klein et al. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

Yann LeCun. A path towards autonomous machine intelligence. *OpenReview preprint*, 2022.

Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

Changliu Liu, Tomer Arnon, Christopher Lazarus, Christopher Strong, Clark Barrett, and Mykel J. Kochenderfer. Algorithms for verifying deep neural networks. *Foundations and Trends in Optimization*, 4(3–4):244–404, 2021.

Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.

Aleksandar Nanevski, Ruy Ley-Wild, Ilya Sergey, and Germán Andrés Morales. Communicating state transition systems for fine-grained concurrent resources.

In *European Symposium on Programming (ESOP)*, 2014. FCSL-PCM library: `https://github.com/imdea-software/fcsl-pcm`.

Peter W. O'Hearn. Separation logic. *Communications of the ACM*, 62(2):86–95, 2019.

Christine Paulin-Mohring. Introduction to the calculus of inductive constructions. In David Delahaye and Bruno Woltzenlogel Paleo, editors, *All about Proofs, Proofs for All*. College Publications, 2015.

John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74, 2002.

Stuart Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering atari, go, chess and shogi by planning with a learned model. In *Nature*, volume 588, pages 604–609, 2020.

Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. Formal specification for deep neural networks. In *Automated Technology for Verification and Analysis (ATVA)*, pages 20–34, 2018.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning (ICML)*, pages 216–224, 1990.

The Coq/Rocq Development Team. The rocq proof assistant reference manual. `https://rocq-prover.org/`, 2024.

Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.